Politechnika Opolska
Wydział Informatyki

Rok akademicki	2025/2026
Przedmiot	Programowanie IV
Forma zajęć	Laboratorium
Prowadzący zajęcia	Dr. Hab. Inż. Dawid Wajnert
Nr grupy	1

Aplikacja desktopowa „Katalog gier komputerowych” w technologii WPF, C# i SQLite.

Nazwisko i imię	Nr indeksu
Moskal Arkadiusz	**107224**

1. Cel i zakres projektu

Celem projektu jest przygotowanie prostej aplikacji desktopowej służącej do prowadzenia katalogu gier komputerowych. Program pozwala zapisywać informacje o grach, takie jak tytuł, okładka w formacie PNG, ocena, krótka recenzja oraz status ukończenia. Dzięki temu użytkownik może uporządkować własną listę gier, rozróżnić tytuły planowane, aktualnie ogrywane, ukończone i porzucone oraz szybko wrócić do zapisanych opinii.

Aplikacja została wykonana jako program okienkowy WPF. Interfejs składa się z głównego okna z listą kart gier, osobnego formularza dodawania i edycji oraz okna szczegółów. Dane są zapisywane w lokalnej bazie SQLite w pliku games.db. Oznacza to, że po ponownym uruchomieniu aplikacji wcześniej dodane gry mogą zostać odczytane z bazy.

Projekt obejmuje podstawowe operacje CRUD: dodawanie nowych rekordów, odczytywanie listy gier, modyfikowanie istniejących wpisów i usuwanie wybranej gry. Dodatkowo dodano filtrowanie według statusu oraz sortowanie według nazwy, oceny albo statusu, co ułatwia korzystanie z katalogu przy większej liczbie pozycji.

2. Technologie i struktura aplikacji

Warstwa wizualna aplikacji została opisana w plikach XAML. Pliki te definiują wygląd okien, rozmieszczenie przycisków, pól tekstowych, list rozwijanych i kart gier. Logika działania znajduje się w plikach C# powiązanych z oknami .xaml.cs. Po kliknięciu przycisku wykonywana jest odpowiednia metoda obsługująca zdarzenie, na przykład dodanie gry, otwarcie szczegółów albo odświeżenie listy.

Projekt korzysta z pakietu Microsoft.Data.Sqlite, który umożliwia połączenie z lokalną bazą SQLite. Za wszystkie operacje bazodanowe odpowiada klasa DbManager. Dzięki wydzieleniu tej klasy kod okien nie musi bezpośrednio zawierać poleceń SQL, a odpowiedzialność za zapis i odczyt danych jest uporządkowana.

Najważniejsze pliki projektu to: MainWindow.xaml i MainWindow.xaml.cs dla głównego widoku, GameFormWindow.xaml i GameFormWindow.xaml.cs dla formularza dodawania i edycji, GameDetailsWindow.xaml i GameDetailsWindow.xaml.cs dla widoku szczegółów, Game.cs dla modelu danych oraz DbManager.cs dla obsługi bazy SQLite.

3. Model danych gry (Game.cs)

Pojedyncza gra jest reprezentowana przez klasę Game. Klasa przechowuje identyfikator rekordu, tytuł, opcjonalną tablicę bajtów z obrazem okładki, opcjonalną ocenę, tekst recenzji oraz liczbowy status ukończenia. Identyfikator jest potrzebny głównie podczas edycji i usuwania, ponieważ pozwala wskazać konkretny rekord w tabeli bazy danych.

W modelu znajdują się również proste właściwości pomocnicze. ScoreText zamienia wartość oceny na tekst wyświetlany w interfejsie. Jeżeli ocena nie została wpisana, aplikacja pokazuje informację „No score", a jeżeli ocena istnieje, dopisuje do niej format /10. Dzięki temu widok nie musi sam sprawdzać, czy ocena jest pusta.

Druga właściwość pomocnicza, CompletionStatus, tłumaczy liczbę statusu na czytelny opis. Wartość 0 oznacza „Plan to play", 1 oznacza „Playing", 2 oznacza „Completed", a 3 oznacza „Dropped". Takie rozwiązanie pozwala przechowywać w bazie prostą liczbę, ale w interfejsie pokazywać zrozumiały tekst.

```
namespace KatalogGierKomp
{
    18 references
    public class Game
    {
        5 references
        public int Id { get; set; }
        8 references
        public byte[]? Image { get; set; }
        8 references
        public string Title { get; set; } = "";
        13 references
        public int? Score { get; set; }
        5 references
        public string Review { get; set; } = "";
        13 references
        public int Completion { get; set; }

        0 references
        public string ScoreText
        {
            get
            {
                if (Score == null)
                {
                    return "No score";
                }

                return Score + "/10";
            }
        }

        0 references
        public string CompletionStatus
        {
            get
            {
                if (Completion == 0) return "Plan to play";
                if (Completion == 1) return "Playing";
                if (Completion == 2) return "Completed";
                if (Completion == 3) return "Dropped";

                return "Unknown";
            }
        }
    }
}
```

4. Obsługa bazy danych (DbManager.cs)

Klasa DbManager tworzy połączenie z bazą SQLite przez connection string Data Source=games.db. Przy uruchomieniu głównego okna wywoływana jest metoda Initialize, która tworzy tabelę games, jeżeli taka tabela jeszcze nie istnieje. Dzięki temu program może działać także przy pierwszym uruchomieniu, kiedy plik bazy danych nie został wcześniej przygotowany ręcznie.

```
1 reference
public void Initialize()
{
    using var connection = new SqliteConnection(ConnectionString);
    connection.Open();

    using var command = connection.CreateCommand();
    command.CommandText =
    @"
    CREATE TABLE IF NOT EXISTS games (
        id_game INTEGER PRIMARY KEY,
        title TEXT,
        image BLOB,
        score INTEGER,
        review TEXT,
        completion INTEGER
    );
    ";
    command.ExecuteNonQuery();
}
```

Tabela przechowuje kolumny odpowiadające właściwościom klasy Game: identyfikator, tytuł, obraz, ocenę, recenzję i status ukończenia.

Metoda LoadGames odczytuje rekordy z tabeli i tworzy z nich listę obiektów Game. Przy odczycie sprawdzane są wartości puste, na przykład brak obrazu lub brak oceny. Metody AddGame, EditGame i DeleteGame wykonują odpowiednio polecenia INSERT, UPDATE i DELETE. W zapytaniach użyto parametrów, takich jak $title i $score, co jest bezpieczniejszym i czytelniejszym sposobem przekazywania danych do SQL.

Przykładowe metody: LoadGame oraz AddGame:

```
1 reference
public List<Game> LoadGames()
{
    List<Game> games = new List<Game>();

    using var connection = new SqliteConnection(ConnectionString);
    connection.Open();

    using var command = connection.CreateCommand();
    command.CommandText =
    @"
    SELECT id_game, title, image, score, review, completion
    FROM games
    ORDER BY id_game DESC;
    ";

    using var reader = command.ExecuteReader();
    while (reader.Read())
    {
        games.Add(new Game
        {
            Id = reader.GetInt32(0),
            Title = reader.IsDBNull(1) ? "" : reader.GetString(1),
            Image = reader.IsDBNull(2) ? null : reader.GetFieldValue<byte[]>(2),
            Score = reader.IsDBNull(3) ? null : reader.GetInt32(3),
            Review = reader.IsDBNull(4) ? "" : reader.GetString(4),
            Completion = reader.IsDBNull(5) ? 0 : reader.GetInt32(5)
        });
    }
```

```
1 reference
public void AddGame(Game game)
{
    using var connection = new SqliteConnection(ConnectionString);
    connection.Open();

    using var command = connection.CreateCommand();
    command.CommandText =
    @"
    INSERT INTO games (title, image, score, review, completion)
    VALUES ($title, $image, $score, $review, $completion);
    ";
    command.Parameters.AddWithValue("$title", game.Title);
    command.Parameters.AddWithValue("$image", game.Image is null ? DBNull.Value : game.Image);
    command.Parameters.AddWithValue("$score", game.Score is null ? DBNull.Value : game.Score);
    command.Parameters.AddWithValue("$review", game.Review);
    command.Parameters.AddWithValue("$completion", game.Completion);
    command.ExecuteNonQuery();
}
```

5. Zapis plików PNG w bazie danych

Obraz okładki jest zapisywany jako BLOB, czyli dane binarne. Zawartość pliku PNG jest zamieniana na tablicę bajtów i zapisywana bezpośrednio w bazie danych. Odpowiada za to metoda ImageToByteArray z klasy Utility. Metoda dostaje otwarty plik jako FileStream,

kopiuje jego zawartość do MemoryStream, a następnie zwraca ją jako byte[]. Ta tablica bajtów trafia do właściwości Image w obiekcie Game.

Podczas zapisu gry metoda AddGame albo EditGame z klasy DbManager przekazuje game.Image do parametru $image. Jeżeli obraz został wybrany, do bazy zapisywana jest tablica bajtów, a jeżeli obraz nie istnieje, zapisywana jest wartość NULL. W bazie dane trafiają do kolumny image w tabeli games.

Przy odczycie działa to odwrotnie. Metoda LoadGames z klasy DbManager pobiera kolumnę image z bazy. Jeśli kolumna nie jest pusta, używa reader.GetFieldValue<byte[]>(2) i zapisuje wynik w Image obiektu Game. Potem metoda ByteArrayToBitmapImage z klasy Utility może zamienić tę tablicę bajtów z powrotem na obraz, tworząc z niej MemoryStream i ustawiając go jako źródło dla BitmapImage.

```
using System.IO;
using System.Windows.Media.Imaging;

namespace KatalogGierKomp
{
    2 references
    internal class Utility
    {
        1 reference
        public static byte[] ImageToByteArray(FileStream file)
        {
            using MemoryStream ms = new MemoryStream();
            file.CopyTo(ms);
            byte[] imageBytes = ms.ToArray();

            return imageBytes;
        }

        1 reference
        public static BitmapImage ByteArrayToBitmapImage(byte[] byteArray)
        {
            MemoryStream memory = new MemoryStream(byteArray);
            BitmapImage imgSource = new BitmapImage();
            imgSource.BeginInit();
            imgSource.StreamSource = memory;
            imgSource.EndInit();

            return imgSource;
        }
    }
}
```

Klasa ImageBytesConverter służy do zamiany obrazu zapisanego jako byte[] na format możliwy do wyświetlenia w WPF. Implementuje interfejs IValueConverter, dlatego może być używana w bindingu w plikach XAML. Metoda Convert dostaje wartość z właściwości Image obiektu Game. Jeżeli wartość nie jest tablicą bajtów albo tablica jest pusta, zwracane jest null,

więc obraz nie jest wyświetlany. Jeżeli dane istnieją, metoda wywołuje Utility.ByteArrayToBitmapImage(bytes), która zamienia bajty na BitmapImage. Dzięki temu kontrolka Image może pokazać okładkę gry. Metoda ConvertBack musi istnieć, bo wymaga tego IValueConverter, ale w tym programie nie jest używana, ponieważ konwersja działa tylko w jedną stronę: z danych binarnych na obraz do wyświetlenia.

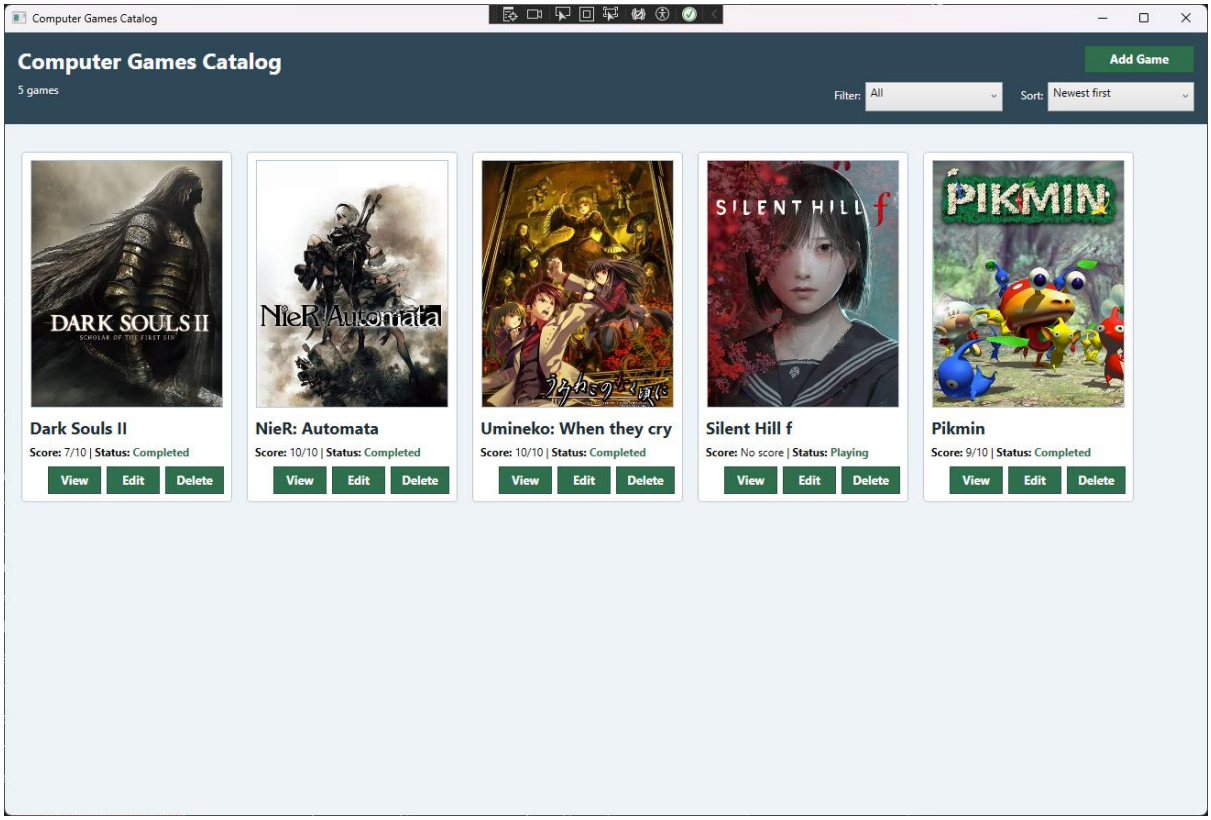
```
using System.Globalization;
using System.Windows.Data;

namespace KatalogGierKomp
{
    0 references
    public class ImageBytesConverter : IValueConverter
    {
        0 references
        public object? Convert(object value, Type targetType, object parameter, CultureInfo culture)
        {
            if (value is not byte[] bytes || bytes.Length == 0)
            {
                return null;
            }

            return Utility.ByteArrayToBitmapImage(bytes);
        }

        0 references
        public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
        {
            throw new NotSupportedException();
        }
    }
}
```

6. Główne okno programu



Po uruchomieniu programu tworzony jest obiekt DbManager, inicjalizowana jest baza danych, a następnie wywoływana jest metoda ShowGames. Główne okno zawiera nagłówek z liczbą wyświetlonych gier, przycisk dodawania, listę rozwijaną do filtrowania oraz listę rozwijaną do sortowania.

```
0 references
public MainWindow()
{
    InitializeComponent();
    dbManager.Initialize();
    ShowGames();
}
```

Poniżej znajduje się lista kart gier z okładką, tytułem, oceną, statusem i przyciskami akcji. Lista gier jest prezentowana przez ItemsControl, a karty układane są w WrapPanel. Oznacza to, że kolejne elementy układają się obok siebie, a po zapełnieniu szerokości okna przechodzą do następnego wiersza. Każda karta ma przypisane przyciski View, Edit i Delete, które otrzymują wybraną grę przez CommandParameter.

Metoda ShowGames odpowiada nie tylko za pokazanie danych, ale także za filtrowanie i sortowanie. Najpierw pobierana jest pełna lista z bazy, później program sprawdza wybrany status w polu filtra, a następnie stosuje wybrane sortowanie. Na końcu lista zostaje przypisana do GamesList.ItemsSource, aktualizowana jest liczba gier i w razie pustego wyniku pokazywany jest komunikat o braku rekordów.

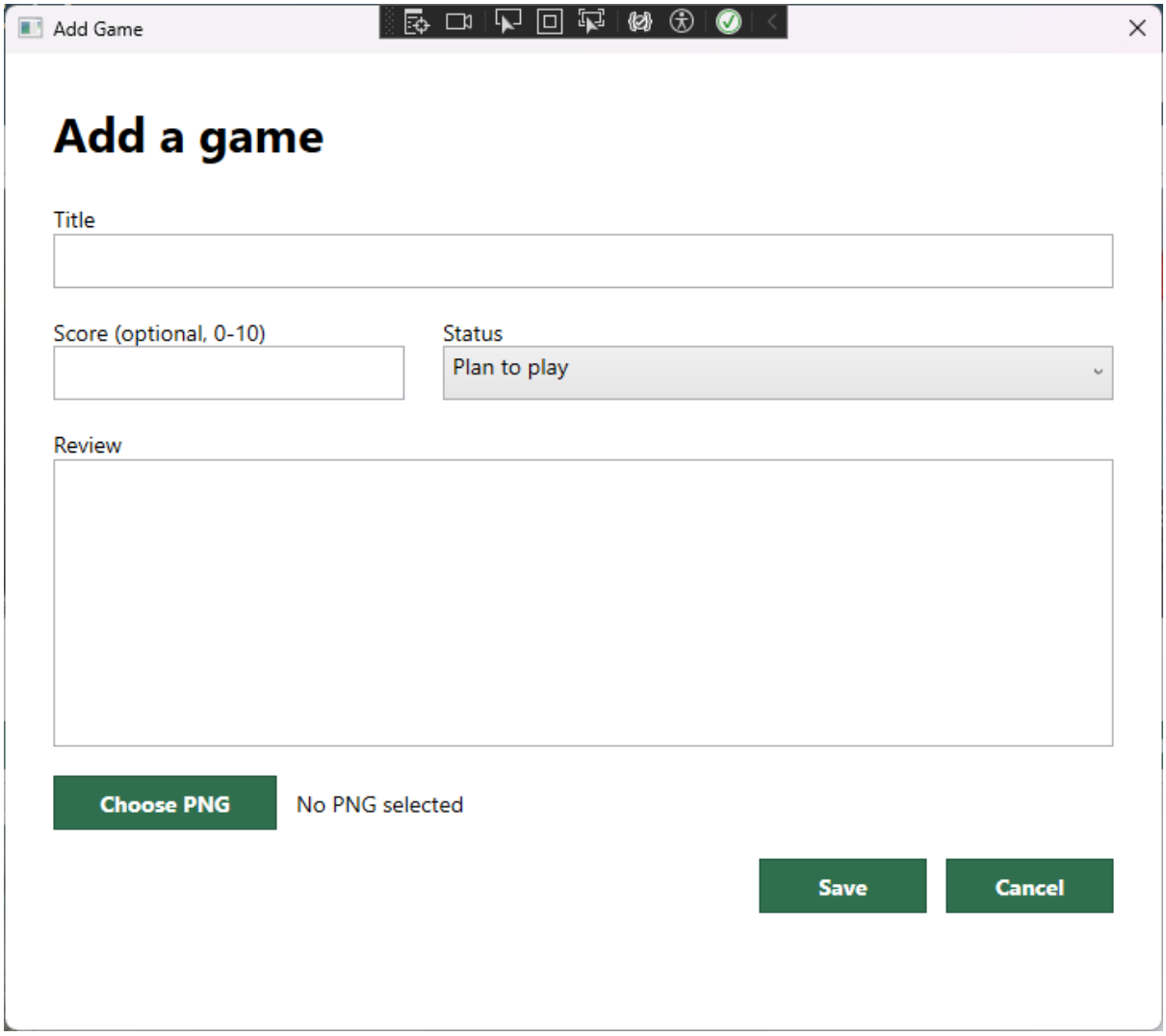
```
5 references
private void ShowGames()
{
    List<Game> games = dbManager.LoadGames();

    int status = FilterBox.SelectedIndex - 1;
    if (status >= 0)
    {
        games = games.Where(game => game.Completion == status).ToList();
    }

    switch (SortBox.SelectedIndex)
    {
        case 1:
            games = games.OrderBy(game => game.Title).ToList();
            break;
        case 2:
            games = games.OrderByDescending(game => game.Title).ToList();
            break;
        case 3:
            games = games.OrderBy(game => game.Score == null)
                          .ThenByDescending(game => game.Score)
                          .ToList();
            break;
        case 4:
            games = games.OrderBy(game => game.Score == null)
                          .ThenBy(game => game.Score)
                          .ToList();
            break;
        case 5:
            games = games.OrderBy(game => game.Completion).ToList();
            break;
    }

    GamesList.ItemsSource = games;
    GameCountText.Text = $"{games.Count} {(games.Count == 1 ? "game" : "games")}";
    EmptyMessage.Visibility = games.Count == 0 ? Visibility.Visible : Visibility.Collapsed;
}
```

7. Dodawanie i edycja gier



Do dodawania i edytowania gry służy to samo okno GameFormWindow. W trybie dodawania pola formularza są puste, a przycisk zapisuje nową pozycję. W trybie edycji konstruktor otrzymuje istniejący obiekt Game, uzupełnia pola danymi tej gry, zmienia tytuł okna na Edit Game i zmienia tekst przycisku na Update.

Formularz zawiera pole tytułu, opcjonalną ocenę od 0 do 10, listę statusów, pole recenzji oraz przycisk wyboru pliku PNG. Po kliknięciu Choose PNG otwierane jest okno wyboru pliku, które dopuszcza tylko obrazy PNG. Wybrany plik jest odczytywany jako strumień i zamieniany na tablicę bajtów przez funkcję pomocniczą.

Najważniejszą częścią formularza jest metoda SaveButton_Click. Najpierw usuwa ona zbędne spacje z tytułu i recenzji, następnie sprawdza, czy tytuł nie jest pusty. Jeśli użytkownik wpisał ocenę, program próbuje zamienić ją na liczbę całkowitą i sprawdza, czy mieści się w zakresie

od 0 do 10. Dopiero po poprawnej walidacji tworzony jest obiekt GameResult, a okno zwraca DialogResult = true.

```
1 reference
private void SaveButton_Click(object sender, RoutedEventArgs e)
{
    string title = TitleBox.Text.Trim();
    string review = ReviewBox.Text.Trim();

    if (string.IsNullOrWhiteSpace(title))
    {
        MessageBox.Show(this, "Enter a title.", "Missing title", MessageBoxButton.OK, MessageBoxImage.Warning);
        TitleBox.Focus();
        return;
    }

    int? score = null;
    string scoreText = ScoreBox.Text.Trim();

    if (!string.IsNullOrWhiteSpace(scoreText))
    {
        if (!int.TryParse(scoreText, out int parsedScore) || parsedScore < 0 || parsedScore > 10)
        {
            MessageBox.Show(this, "Enter a score from 0 to 10, or leave it empty.", "Invalid score", MessageBoxButton.OK, MessageBoxImage.Warning);
            ScoreBox.Focus();
            return;
        }

        score = parsedScore;
    }

    GameResult = new Game
    {
        Id = editedGame?.Id ?? 0,
        Title = title,
        Image = selectedImageBytes ?? editedGame?.Image,
        Score = score,
        Review = review,
        Completion = CompletionBox.SelectedIndex
    };

    DialogResult = true;
}
```

W samym GameFormWindow gra nie jest jeszcze zapisywana do bazy. Formularz tylko przygotowuje obiekt GameResult z danymi wpisanymi przez użytkownika. Dopiero w MainWindow.xaml.cs, po zamknięciu formularza, aplikacja sprawdza, czy GameResult istnieje, a następnie wywołuje dbManager.AddGame(window.GameResult) przy dodawaniu nowej gry albo dbManager.EditGame(window.GameResult) przy edycji. Po tej operacji metoda ShowGames() odświeża listę gier w głównym oknie.

```
1 reference
private void AddGameButton_Click(object sender, RoutedEventArgs e)
{
    GameFormWindow window = new GameFormWindow();
    window.Owner = this;

    if (window.ShowDialog() == true && window.GameResult != null)
    {
        dbManager.AddGame(window.GameResult);
        ShowGames();
    }
}

1 reference
private void EditGameButton_Click(object sender, RoutedEventArgs e)
{
    Button button = (Button)sender;
    Game game = (Game)button.CommandParameter;

    GameFormWindow window = new GameFormWindow(game);
    window.Owner = this;

    if (window.ShowDialog() == true && window.GameResult != null)
    {
        dbManager.EditGame(window.GameResult);
        ShowGames();
    }
}
```

8. Podgląd szczegółów gry

Okno GameDetailsWindow służy do pokazania jednej gry w większym i bardziej czytelnym układzie. Po otwarciu okna do właściwości DataContext przypisywany jest wybrany obiekt Game. Dzięki temu elementy XAML mogą pobierać wartości przez binding, na przykład tytuł, ocenę, status, recenzję i obraz okładki.

W widoku szczegółów okładka ma większy rozmiar niż na karcie w głównej liście, a obok niej znajduje się pełny opis gry. Dłuższa recenzja jest umieszczona w obszarze przewijanym, dlatego nie musi mieścić się w całości na ekranie. Przycisk Close wywołuje prostą metodę zamykającą okno.

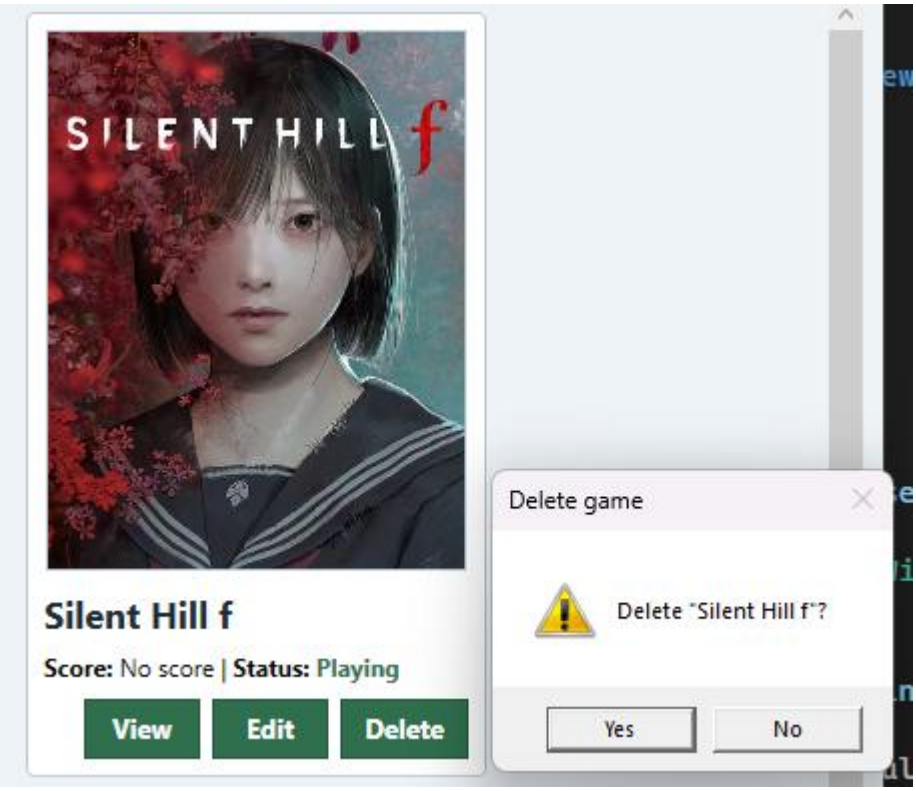
```
using System.Windows;

namespace KatalogGierKomp
{
    4 references
    public partial class GameDetailsWindow : Window
    {
        1 reference
        public GameDetailsWindow(Game game)
        {
            InitializeComponent();
            DataContext = game;
        }

        1 reference
        private void CloseButton_Click(object sender, RoutedEventArgs e)
        {
            Close();
        }
    }
}
```

9. Przykładowe użycie aplikacji

Podstawowy przepływ działania zaczyna się w konstruktorze MainWindow. Program inicjalizuje komponenty okna, przygotowuje tabelę w bazie danych i ładuje gry. Po dodaniu, edycji albo usunięciu rekordu ponownie wywoływana jest metoda ShowGames, dlatego użytkownik od razu widzi aktualny stan katalogu.

Przy dodawaniu gry główne okno otwiera formularz jako okno dialogowe. Jeżeli użytkownik kliknie Save i walidacja przejdzie poprawnie, formularz zwraca gotowy obiekt GameResult. Główne okno zapisuje ten obiekt w bazie przez DbManager.AddGame. Podobny mechanizm działa przy edycji, ale formularz dostaje już istniejącą grę i po zapisie wywoływana jest metoda EditGame.

Usuwanie gry wymaga potwierdzenia w oknie MessageBox. To zabezpiecza użytkownika przed przypadkowym kliknięciem przycisku Delete. Dopiero po wybraniu odpowiedzi Yes program usuwa rekord z bazy i odświeża listę.